



Tarea 01

Introducción y problemas básicos

No. de cuenta	Nombre
320325775	Hernández Vázquez Carlos Arturo
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Índice

1	Ejercicio 1	2
1.1	Solución	2
2	Ejercicio 2	7
2.1	Solución	8
3	Ejercicio 3	10
3.1	Solución	10
4	Ejercicio 4	13
4.1	Solución	13
5	Ejercicio 5	19
5.1	Solución	19
6	Ejercicio 6	22
6.1	Solución	22
7	Ejercicio 7	24
7.1	Solución	24
8	Referencias Bibliográficas	26

Ejercicio 1

Enunciado

Se sabe que un algoritmo de ordenamiento Q con complejidad $O(f(n))$ y tiempo de procesamiento $T(n) = cf(n)$, tarda 0.4 segundos en ejecutarse cuando la lista a ordenar tiene 10,000 elementos.

1. Estima el tiempo de ejecución para una lista de 60,000 elementos.
2. Estima el tiempo de ejecución para una lista de 1,500,000 elementos.
3. Determina el tamaño máximo de la lista que puede ser ordenada en menos de 10 segundos.

Puedes asumir que el tiempo de ejecución del algoritmo Q tiene una complejidad $O(n \log n)$

Solución

1. El **primer punto** pide estimar el tiempo de ejecución del algoritmo Q cuando la lista a ordenar tiene $n = 60,000$ elementos.

La complejidad del algoritmo es $O(n \log n)$ y el tiempo real se modela como:

$$T(n) = c \cdot n \log(n)$$

Utilizamos el caso base proporcionado para determinar el valor de c : con $n = 10,000$, el tiempo es $T(10,000) = 0.4$ segundos.

Elección de la Base del Logaritmo

Antes de continuar creemos necesario señalar que **la base que elijamos no afecta en absoluto a las predicciones de tiempo**. Esto ocurre porque el cambio de base de un logaritmo es una conversión lineal que la constante c absorbe por completo.

Demostración formal:

Si cambiamos el logaritmo de una base a a una base b , usamos la propiedad:

$$\log_a(n) = \frac{\log_b(n)}{\log_b(a)}$$

Si planteamos la ecuación con la base a :

$$T(n) = c_a \cdot n \log_a(n) = c_a \cdot n \frac{\log_b(n)}{\log_b(a)} = \left(\frac{c_a}{\log_b(a)} \right) \cdot n \log_b(n)$$

Definiendo una nueva constante $c_b = \frac{c_a}{\log_b(a)}$, la ecuación se convierte en:

$$T(n) = c_b \cdot n \log_b(n)$$

Al despejar la constante utilizando el caso base $T(10,000) = 0.4$ s, el valor de c se ajustará automáticamente según la base que elijamos, y cualquier cálculo posterior para otro valor de n dará exactamente el mismo tiempo en segundos.

Encontrar el valor de la contante c

Usando logaritmo en base 10 (por simplicidad de cálculo manual).

Sustituimos los valores en el modelo:

$$0.4 = c \cdot 10,000 \cdot \log_{10}(10,000)$$

$$0.4 = c \cdot (10,000 \cdot 4)$$

$$0.4 = c \cdot 40,000$$

$$c = \frac{0.4}{40,000} = 10^{-5} = 0.00001$$

Estimar el tiempo para $n = 60,000$ elementos

Ahora que conocemos la constante c , evaluamos el tiempo de ejecución en $n = 60,000$.

Sustituimos en la ecuación:

$$T(60,000) = 10^{-5} \cdot 60,000 \cdot \log_{10}(60,000)$$

$$T(60,000) = 0.6 \cdot \log_{10}(60,000)$$

Calculamos $\log_{10}(60,000)$:

$$\log_{10}(60,000) = \log_{10}(6 \cdot 10^4) = \log_{10}(6) + \log_{10}(10^4) \approx 0.77815 + 4 = 4.77815$$

Multiplicamos:

$$T(60,000) \approx 0.6 \cdot 4.77815 \approx \mathbf{2.867 \text{ segundos}}$$

Conclusión:

El tiempo estimado de ejecución para una lista de 60,000 elementos es de aproximadamente **2.867 segundos** (o 2.87 segundos si redondeamos a dos decimales).

2. Para resolver el **segundo punto** debemos estimar el tiempo de ejecución del algoritmo Q cuando la lista tiene: $n = 1,500,000$ **elementos**.

Utilizaremos el valor de la constante c que calculamos y la base logarítmicas 10.

Valor de la constante c

Sabemos que el tiempo de ejecución del algoritmo se modela como:

$$T(n) = c \cdot n \log(n)$$

Donde, a partir de nuestro caso base ($T(10,000) = 0.4$ s) y utilizando logaritmo en base 10:

$$c = 10^{-5} = 0.00001$$

Estimar el tiempo para $n = 1,500,000$ elementos

Sustituimos $n = 1,500,000$ y $c = 10^{-5}$ en nuestra ecuación:

$$T(1,500,000) = 10^{-5} \cdot 1,500,000 \cdot \log_{10}(1,500,000)$$

Simplificamos el producto de la constante c por n :

$$10^{-5} \cdot 1,500,000 = 15$$

Ahora, calculamos $\log_{10}(1,500,000)$.

$$\log_{10}(1,500,000) = \log_{10}(1.5 \cdot 10^6) = \log_{10}(1.5) + \log_{10}(10^6)$$

Sabiendo que $\log_{10}(1.5) \approx 0.17609$ y $\log_{10}(10^6) = 6$:

$$\log_{10}(1,500,000) \approx 0.17609 + 6 = 6.17609$$

Multiplicamos por el factor restante:

$$T(1,500,000) \approx 15 \cdot 6.17609 \approx \mathbf{92.641 \text{ segundos}}$$

Conclusión:

El tiempo estimado de ejecución para una lista de 1,500,000 elementos es de aproximadamente **92.64 segundos** (o **1 minuto y 32.64 segundos**).

Este resultado refleja el comportamiento de la complejidad $O(n \log n)$: aunque el tamaño de la lista aumentó **150 veces** (de 10,000 a 1,500,000), el tiempo de procesamiento solo aumentó unas **231 veces** (de 0.4 s a 92.64 s).

3. Para resolver el **tercer punto** de manera exacta, utilizaremos la **función W de Lambert** [1]. Este método evita las aproximaciones por tanteo y proporciona una solución analítica cerrada para la ecuación.

1. Formulación de la Desigualdad

El objetivo es encontrar el tamaño máximo de la lista n tal que el tiempo de ejecución del algoritmo sea estrictamente menor a 10 segundos:

$$T(n) < 10$$

A partir de los resultados obtenidos con el caso base $T(10,000) = 0.4$ s, podemos plantear la ecuación utilizando cualquier base logarítmica.

Usamos Base 10 (donde $c = 10^{-5}$):

$$10^{-5} \cdot n \log_{10}(n) < 10 \implies n \log_{10}(n) < 10^6$$

Cambiando el logaritmo a base natural ($\log_{10}(n) = \frac{\ln(n)}{\ln(10)}$):

$$n \frac{\ln(n)}{\ln(10)} < 10^6 \implies n \ln(n) < 10^6 \ln(10)$$

La ecuación que define el límite exacto es:

$$n \ln(n) = 10^6 \ln(10)$$

2. Transformación a la Forma de Lambert

La **función W de Lambert** es la función inversa de $f(w) = we^w$. Es decir:

$$we^w = z \iff w = W(z)$$

Para llevar nuestra ecuación $n \ln(n) = 10^6 \ln(10)$ a esta forma exacta, aplicamos la identidad algebraica de la función exponencial, escribiendo n como $e^{\ln(n)}$:

$$e^{\ln(n)} \ln(n) = 10^6 \ln(10)$$

Reordenando los términos del lado izquierdo:

$$\ln(n) \cdot e^{\ln(n)} = 10^6 \ln(10)$$

Esta ecuación ya tiene exactamente la estructura $we^w = z$, donde:

- La variable incógnita es $w = \ln(n)$
- El término constante es $z = 10^6 \ln(10)$

3. Aplicación de la Función W de Lambert

Aplicamos la función W (específicamente la rama principal W_0 , ya que nuestro argumento z es positivo) en ambos lados de la ecuación para despejar $\ln(n)$, es decir:

Nota

La **función W de Lambert** se define matemáticamente con un único propósito: ser la función inversa de la expresión $f(x) = xe^x$. Es decir, W es la herramienta diseñada para “deshacer” la operación de multiplicar una variable por su propia exponencial.

Por la propiedad de cualquier función inversa, sabemos que al aplicar la inversa a la función original, estas se cancelan dejando únicamente el argumento:

$$f^{-1}(f(x)) = x$$

Si aplicamos esto a nuestra definición:

- (a) La función directa es: $f(x) = xe^x$
- (b) La función inversa es: $f^{-1}(z) = W(z)$
- (c) Al componerlas, obtenemos la identidad fundamental de Lambert:

$$W(x \cdot e^x) = x$$

$$W(\ln(n) \cdot e^{\ln(n)}) = W(10^6 \ln(10))$$

$$\ln(n) = W(10^6 \ln(10))$$

Para despejar finalmente n , aplicamos la función exponencial (base e) a ambos miembros:

$$n = e^{W(10^6 \ln(10))}$$

Simplificar la expresión

Existe una propiedad fundamental de la función W de Lambert que establece que:

$$W(z) \cdot e^{W(z)} = z \implies e^{W(z)} = \frac{z}{W(z)}$$

Sustituyendo esta propiedad en nuestra solución para n (donde $z = 10^6 \ln(10)$), reducimos la expresión exponencial a una división mucho más sencilla:

$$n = \frac{10^6 \ln(10)}{W(10^6 \ln(10))}$$

4. Evaluación Numérica de la Solución

Ahora evaluamos los valores numéricos:

- (a) Calculamos el argumento z :

$$z = 10^6 \ln(10) \approx 1,000,000 \cdot 2.30258509 = 2,302,585.09$$

- (b) Evaluamos la función W_0 de Lambert para este argumento. Dado que es una función trascendente, su valor se obtiene mediante aproximaciones numéricas de alta precisión (por ejemplo, con métodos de Newton-Raphson, librerías científicas como SciPy o en este caso utilizamos WolframAlpha [2]):

$$W_0(2,302,585.09) = \text{ProductLog}W(10^6 \ln(10))$$

$$W_0(2,302,585.09) \approx 12.151944$$

Nota: La función W de Lambert aparece como `ProductLog()` en **WolframAlpha**.

- (c) Calculamos el valor final de n :

$$n \approx \frac{2,302,585.09}{12.151944} \approx 189,481.28$$

5. Conclusión

Dado que n representa el número de elementos físicos de una lista, este debe ser necesariamente un **número entero**.

- Para $n = 189,481$, el tiempo estimado de ejecución es de **9.99998 segundos** (cumple estrictamente con $T(n) < 10 \implies n \log_{10}(n) < 10^6$).
- Para $n = 189,482$, el tiempo estimado de ejecución es de **10.00004 segundos** (excede el límite de 10 segundos).

Por lo tanto, la solución exacta y justificada analíticamente para el tamaño máximo de la lista es:

$$\mathbf{n = 189,481 \text{ elementos}}$$

Ejercicio 2

Enunciado

¿Cuáles de las siguientes afirmaciones son verdades y cuáles falsas? Justifica tu respuesta en cada caso

1. Si $f(n) = n^k$ y $g(n) = c^n$, para algunas constantes $c > 1$, $k \geq 0$, entonces $g(n) = \Omega(f(n))$
2. Si $f(n) = n!$ y $g(n) = 2^n$, $f(n) = O(g(n))$
3. Si $f(n) = O(n)$ y $g(n) = \Theta(n)$, entonces:
 - $f(n) + g(n) = O(n)$
 - $f(n)g(n) = O(n^2)$
 - $f(n) - g(n) = O(1)$
4. Existen funciones f, g tales que no se cumple que $f(n) = O(g(n))$ ni $f(n) = \Omega(g(n))$

Solución

¿Cuáles de las siguientes afirmaciones son verdaderas y cuáles falsas? Justifica tu respuesta en cada caso:

1. Si $f(n) = n^k$ y $g(n) = c^n$, para algunas constantes $c > 1$, $k \geq 0$, entonces $g(n) = \Omega(f(n))$.

La afirmación es verdadera.

Evaluamos el siguiente límite aplicando la regla de L'Hôpital k veces:

$$\lim_{n \rightarrow \infty} \frac{n^k}{c^n} = \lim_{n \rightarrow \infty} \frac{k!}{c^n (\ln c)^k} = 0$$

donde el numerador es una constante finita que depende únicamente de k , mientras que el denominador tiende a infinito debido al factor c^n con $c > 1$.

Por definición (en forma de límite) de ω -notación

$$g(n) = \omega(f(n)) \implies g(n) = \Omega(f(n))$$

2. Si $f(n) = n!$ y $g(n) = 2^n$, entonces $f(n) = O(g(n))$.

La afirmación es falsa.

Expandimos el cociente y observamos que $\frac{n}{2} \geq 2$ para todo $n \geq 4$:

$$\frac{n!}{2^n} = \left(\frac{1}{2} \cdot \frac{2}{2} \cdot \frac{3}{2}\right) \left(\frac{4}{2} \cdot \frac{5}{2} \cdots \frac{n}{2}\right) \geq \frac{3}{4} \cdot 2^{n-3}$$

Evaluamos el límite

$$\lim_{n \rightarrow \infty} \left(\frac{3}{4} \cdot 2^{n-3}\right) = \frac{3}{4} \lim_{n \rightarrow \infty} 2^{n-3} = \infty$$

Usando límite evaluado y la desigualdad planteada tenemos que

$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \infty \implies f(n) = \omega(g(n)) \implies f(n) \neq O(g(n))$$

3. Si $f(n) = O(n)$ y $g(n) = \Theta(n)$, entonces:

- $f(n) + g(n) = O(n)$

La afirmación es verdadera.

Por definición de $f(n) = O(n)$, existen constantes $c_1 > 0$ y $n_1 \in \mathbb{N}$ tales que:

$$0 \leq f(n) \leq c_1 n, \quad \forall n \geq n_1$$

Por definición de $g(n) = \Theta(n)$, en particular $g(n) = O(n)$, existen constantes $c_2 > 0$ y $n_2 \in \mathbb{N}$ tales que:

$$0 \leq g(n) \leq c_2 n, \quad \forall n \geq n_2$$

Tomando $n_0 = \max(n_1, n_2)$ y la constante $c_3 = c_1 + c_2 > 0$, sumamos ambas desigualdades para todo $n \geq n_0$:

$$0 \leq f(n) + g(n) \leq c_1n + c_2n = (c_1 + c_2)n = c_3n$$

Por tanto, existen $c_3 > 0$ y $n_0 \in \mathbb{N}$ que satisfacen la definición, concluyendo que:

$$f(n) + g(n) = O(n)$$

- $f(n)g(n) = O(n^2)$

La afirmación es verdadera.

Por definición de $f(n) = O(n)$, existen constantes $c_1 > 0$ y $n_1 \in \mathbb{N}$ tales que:

$$0 \leq f(n) \leq c_1n, \quad \forall n \geq n_1$$

Por definición de $g(n) = \Theta(n)$, en particular $g(n) = O(n)$, existen constantes $c_2 > 0$ y $n_2 \in \mathbb{N}$ tales que:

$$0 \leq g(n) \leq c_2n, \quad \forall n \geq n_2$$

Tomando $n_0 = \max(n_1, n_2)$ y la constante $c_3 = c_1c_2 > 0$, multiplicamos ambas desigualdades término a término para todo $n \geq n_0$:

$$0 \leq f(n)g(n) \leq (c_1n)(c_2n) = c_1c_2n^2 = c_3n^2$$

Por tanto, existen $c_3 > 0$ y $n_0 \in \mathbb{N}$ que satisfacen la definición, concluyendo que:

$$f(n)g(n) = O(n^2)$$

- $f(n) - g(n) = O(1)$

La afirmación es falsa.

Consideremos como contraejemplo $f(n) = 5n$ y $g(n) = n$, las cuales satisfacen claramente que $f(n) = O(n)$ y $g(n) = \Theta(n)$.

Tenemos que:

$$f(n) - g(n) = 5n - n = 4n$$

Para que $f(n) - g(n) = O(1)$, deberían existir constantes $c > 0$ y $n_0 \in \mathbb{N}$ tales que:

$$0 \leq 4n \leq c \cdot 1, \quad \forall n \geq n_0 \implies n \leq \frac{c}{4}, \quad \forall n \geq n_0$$

lo cual es una contradicción, pues el conjunto $\{n \in \mathbb{N} \mid n \geq n_0\}$ no está acotado superiormente por ninguna constante $\frac{c}{4}$. Por lo tanto:

$$f(n) - g(n) \neq O(1)$$

4. Existen funciones f, g tales que no se cumple que $f(n) = O(g(n))$ ni $f(n) = \Omega(g(n))$.

La afirmación es verdadera.

El orden asintótico **no** satisface la propiedad de tricotomía y podemos dar un ejemplo donde existen pares de funciones que son asintóticamente incomparables [8].

Consideremos:

$$f(n) = n \quad \text{y} \quad g(n) = n^{1+\sin n}$$

Dado que el valor de $\sin n$ oscila continuamente entre -1 y 1 conforme n tiende a infinito, el exponente de $g(n)$ varía indefinidamente en el intervalo $[0, 2]$.

- $f(n) \neq O(g(n))$: Cuando $\sin n \approx -1$, la función $g(n)$ cae cerca de $n^0 = 1$. Para cualquier constante $c > 0$, la desigualdad $n \leq c \cdot 1$ se vuelve falsa para valores grandes de n . Por lo tanto, $f(n)$ no puede estar acotada por arriba por $c \cdot g(n)$.
- $f(n) \neq \Omega(g(n))$: Cuando $\sin n \approx 1$, la función $g(n)$ se acerca a n^2 . Para cualquier constante fija $c > 0$, la desigualdad $c \cdot n^2 \leq n \implies n \leq \frac{1}{c}$ es falsa para valores grandes de n . Por lo tanto, $f(n)$ no puede ser acotada por abajo por $c \cdot g(n)$.

Se concluye que $f(n) \neq O(g(n))$ y $f(n) \neq \Omega(g(n))$.

Ejercicio 3

Enunciado

Considera los siguientes dos problemas sobre CLANES en gráficas. En el problema de decisión, la pregunta a resolver es si existe o no un clan de cierto tamaño. El otro es el problema de búsqueda, en el que queremos de hecho encontrar cuáles son los vértices que conforman el clan.

Supón que existe un oráculo que nos dice, por el precio de una moneda por pregunta, si una gráfica tiene o no clan de cierto tamaño.

- Demuestra que haciendo una serie de preguntas al oráculo, podemos resolver el problema de optimización utilizando solo una cantidad de monedas que crece polinomialmente como una función del número de vértices.
- Concluye que podemos resolver el problema de decisión en tiempo polinomial sí y sólo sí podemos resolver el problema de búsqueda también en tiempo polinomial.

Solución

Considera los siguientes dos problemas sobre CLANES en gráficas. En el problema de decisión, la pregunta a resolver es si existe o no un clan de cierto tamaño. El otro es el problema de búsqueda, en el que queremos de hecho encontrar cuáles son los vértices que conforman el clan.

Supón que existe un oráculo que nos dice, por el precio de una moneda por pregunta, si una gráfica tiene o no clan de cierto tamaño.

- Demuestra que haciendo una serie de preguntas al oráculo, podemos resolver el problema de optimización utilizando solo una cantidad de monedas que crece polinomialmente como una función del número de vértices.

El costo para cada llamada al oráculo de "una moneda" lo consideramos como una operación de tiempo constante $O(1)$.

Consideramos el siguiente pseudocódigo:

Require: Gráfica $G = (V, E)$

Ensure: k' el tamaño del clan más grande en G

```

1: function TAMANÑOCLANMASGRANDE( $G$ )
2:    $high \leftarrow |V|$ 
3:    $low \leftarrow 0$ 
4:    $k' \leftarrow 0$ 
5:   while  $low \leq high$  do
6:      $middle \leftarrow \lfloor (high + low)/2 \rfloor$ 
7:     if  $G$  tiene un clan de tamaño  $middle$  (oráculo) then
8:        $k' \leftarrow middle$ 
9:        $low \leftarrow middle + 1$ 
10:    else
11:       $high \leftarrow middle - 1$ 
12:    end if
13:  end while
14:  return  $k'$ 
15: end function

```

Proposición 1: Si existe un clan de tamaño k , entonces existe un clan de tamaño $k - 1$.

Demostración: Por contradicción, supongamos que tenemos un clan C de tamaño k , pero no de tamaño $k - 1$. Para un $v \in C$ arbitrario, consideramos la gráfica $C - \{v\}$. Por definición de gráfica completa, cada $v' \in C - \{v\}$ tiene ahora grado $k - 2$, i.e., sigue siendo adyacente a los vértices de $C - \{v\}$, siendo una gráfica completa y por tanto un clan, lo cual es una contradicción. Por tanto, existe un clan de tamaño $k - 1$.

Proposición 2: Si no existe un clan de tamaño k , entonces no existe un clan de tamaño $k + 1$.

Demostración: Por contrapositiva de la proposición 1.

Estas dos proposiciones nos garantizan que el algoritmo anterior solo descarta porciones del arreglo que no nos brindan información para encontrar el clan más grande de tamaño k' . Además, el algoritmo solo es una versión modificada de búsqueda binaria. Entonces, tiene complejidad $O(\log |V|)$.

Teniendo el k' . Consideramos el siguiente pseudocódigo:

Require: Gráfica $G = (V, E)$, k' el tamaño del clan más grande en G

Ensure: G' el clan más grande de G

```

1: function CLANMASGRANDE( $G, k'$ )
2:    $G' \leftarrow G$ 
3:   for all  $v \in V$  do
4:     if  $G' - \{v\}$  tiene un clan de tamaño  $k'$  then
5:        $G' \leftarrow G' - \{v\}$ 
6:     end if
7:   end for
8:   return  $G'$ 
9: end function

```

El algoritmo solo considera en G' los vértices que sí forman parte de un clan de tamaño k' . Para todo vértice que no figura dentro del clan, se descarta usando el oráculo. Al iterar sobre los vértices V , la complejidad del algoritmo es $O(|V|)$

La complejidad conjunta de ambos algoritmos es de $O(\log |V| + |V|) = O(|V|)$.

Por tanto, solo gastando una cantidad polinomial de monedas podemos encontrar el clan de tamaño máximo de una gráfica.

- Concluye que podemos resolver el problema de decisión en tiempo polinomial sí y sólo sí podemos resolver el problema de búsqueda también en tiempo polinomial.

Demostración: Para toda $G = (V, E)$ gráfica

- Si podemos resolver el problema de decisión en tiempo polinomial, entonces podemos resolver el problema de búsqueda también en tiempo polinomial.

Supongamos que el algoritmo de decisión tiene complejidad polinomial: *complejidad_decision*.

Iteramos por cada $n \in \{|V|, |V| - 1, |V| - 2, \dots, 1\}$ y usamos el algoritmo de decisión para saber si existe un clan de tamaño n . Nos quedamos con el primer resultado positivo, digamos n' . Este paso nos toma $O(|V| \cdot \text{complejidad_decision})$, que es un polinomio.

Usando el algoritmo *ClanMasGrande*, pero con el algoritmo de decisión en lugar del oráculo, con G y n' obtenemos el clan respectivo G' . Toma $O(|V| \cdot \text{complejidad_decision})$.

En total, podemos resolver el problema en tiempo polinomial:

$$\begin{aligned}
 &O(|V| \cdot \text{complejidad_decision} + |V| \cdot \text{complejidad_decision}) \\
 &= O(|V| \cdot \text{complejidad_decision})
 \end{aligned}$$

- Si podemos resolver el problema de búsqueda en tiempo polinomial, entonces podemos resolver el problema de decisión también en tiempo polinomial.

Supongamos que el algoritmo de búsqueda tiene complejidad polinomial.

Ejecutamos el algoritmo de búsqueda. Si la respuesta es un clan de tamaño $\geq k$, regresamos 'SÍ' y en otro caso 'NO'. Se justifica por las proposiciones 1 y 2 del inciso anterior. La complejidad final es la misma que la del algoritmo de búsqueda, es decir, polinomial.

Por tanto, podemos resolver el problema de decisión en tiempo polinomial sí y sólo sí podemos resolver el problema de búsqueda también en tiempo polinomial.

Ejercicio 4

Enunciado

Investiga en qué consisten los problemas de:

- Máximo Corte (MAX CUT)
- Cubierta de Vértices
- Conjunto Independiente

Para cada uno de los problemas:

1. Da la definición en su forma canónica, en su versión de decisión.
2. Ilustra un ejemplar concreto pequeño (no trivial), y da la respuesta a la pregunta de decisión. Usar parámetros completamente diferentes para cada problema.
3. Da la definición del problema en su versión de optimización.

Responde las siguientes preguntas sobre la gráfica no dirigida de la Figura 1:

- ¿Tiene un ciclo Hamiltoniano?
- ¿Cuál es el clan más grande?
- ¿Cuál es el conjunto independiente más grande?
- ¿Cuál es la cubierta de vértices más pequeña?
- Suponiendo que todas las aristas tienen peso 1, ¿Cuál es el corte máximo?

Figura 1: Gráfica Ejercicio 4

Solución

Investiga en qué consisten los problemas de:

- Máximo Corte (MAX CUT)
- Cubierta de Vértices
- Conjunto Independiente

Para cada uno de los problemas:

1. Da la definición en su forma canónica, en su versión de decisión.
2. Ilustra un ejemplar concreto pequeño (no trivial), y da la respuesta a la pregunta de decisión. Usar parámetros completamente diferentes para cada problema.

3. Da la definición del problema en su versión de optimización.

1. **Maximo Corte (MAX CUT):**

El problema del CORTE MÁXIMO (MAX-CUT) es uno de los problemas de partición de gráficas más sencillos de conceptualizar, y sin embargo, uno de los problemas de optimización combinatoria más difíciles de resolver. El objetivo de MAX-CUT es particionar el conjunto de vértices de una gráfica en dos subconjuntos, de manera que la suma de los pesos de las aristas con un extremo en cada subconjunto sea máxima. [3]

Definición:

Sea (G, w) una gráfica, donde G es una gráfica y w una función de peso en sus aristas. Un *corte* es una partición $(S, V \setminus S)$ del conjunto de vértices V en las partes S y $V \setminus S$. El peso $w(S, V \setminus S)$ de un corte está dado por

$$w(S, V \setminus S) := \sum_{i \in S, j \in V \setminus S} w(i, j).$$

El corte de peso máximo se denomina *corte máximo* (abreviado como *max-cut*), y su peso se denota por $mc(G, w)$, es decir,

$$mc(G, w) := \max_{S \subseteq V} w(S, V \setminus S).$$

Observamos que permitimos $S = \emptyset$ o $S = V$, es decir, una de las clases de la partición $(S, V \setminus S)$ puede estar vacía. Tal corte puede ser máximo, por ejemplo, cuando todos los pesos $w(i, j)$ son negativos.

Definición en Forma Canónica:

CORTE MÁXIMO (MAX-CUT)

- **Datos:** Una gráfica G , con una función de pesos en sus aristas w , con sus respectivas particiones $(S, V \setminus S)$.
- **Pregunta:** ¿Existe una gráfica (G, w) tal que la partición $(S, V \setminus S)$ resulta en un corte máximo?

Definición en Versión de Decisión:

CORTE MÁXIMO (MAX-CUT)

- **Datos:** Una gráfica $G = (V, E)$, una función de pesos en las aristas w y un número entero k .
- **Pregunta:** ¿Existe una partición $(S, V \setminus S)$ del conjunto de vértices V , para una gráfica G , no dirigida y no vacía, tal que el peso total del corte sea al menos k ? Es decir, ¿se cumple que

$$\sum_{i \in S, j \in V \setminus S} w(i, j) \geq k \quad ?$$

Ejemplar concreto

- **Entrada:** (G, w) , donde $G = (V, E)$ es un gráfica definida por:
 - Conjunto de vértices: $V = \{A, B, C, D, E, F\}$
 - Conjunto de aristas E con su respectiva función de peso w

$$E = \{(A, B), (A, C), (B, D), (C, D), (C, E), (D, F), (E, F)\}$$

donde los pesos para cada arista son:

$$\begin{aligned} w(A, B) = 3, \quad w(A, C) = 5, \quad w(B, D) = 2, \\ w(C, D) = 4, \quad w(C, E) = 6, \quad w(D, F) = 1, \quad w(E, F) = 3 \end{aligned}$$

- **Respuesta:** Si, Partición óptima: $S = \{A, D, E\}$ y $V \setminus S = \{B, C, F\}$, peso total máximo: $mc(G, w) = 3 + 5 + 2 + 4 + 6 + 1 + 3 = 24$

Definición en Forma de Optimización:**CORTE MÁXIMO (MAX-CUT)**

- **Datos:** Una gráfica $G = (V, E)$ y una función de pesos en sus aristas w , y una partición S de G y $V \setminus S$.
- **Pregunta:** ¿Cuál es la partición $(S, V \setminus S)$ del conjunto de vértices V que maximiza el peso total del corte, es decir, que maximiza la expresión?:

$$w(S, V \setminus S) = \sum_{i \in S, j \in V \setminus S} w(i, j)$$

2. Cubierta de Vértices**Definición:**

Sea $G = (V, E)$ una gráfica. Una *cobertura de vértices* de G es un subconjunto W de V tal que cada arista de G es incidente con al menos un vértice en W .

Definición en Forma Canónica:**CUBIERTA DE VÉRTICES**

- **Datos:** Una gráfica $G = (V, E)$ y un entero positivo $k \leq |V|$.
- **Pregunta:** ¿Existe una cobertura o cubierta de vértices $W \subseteq V$ de tamaño $|W| \leq k$, es decir, un subconjunto W tal que para toda arista $\{u, v\} \in E$ se cumpla $u \in W$ o $v \in W$?

Definición en Versión de Decisión:**CUBIERTA DE VÉRTICES**

- **Datos:** Una gráfica $G = (V, E)$ y un entero $k \geq 1$.

- **Pregunta:** ¿Admite G un subconjunto de vértices $W \subseteq V$ tal que $|W| \leq k$ y $W \cap \{u, v\} \neq \emptyset$ para todo $\{u, v\} \in E$?

Ejemplar concreto

- **Entrada:**

- Gráfica $G = (V, E)$, y un entero k :

- * $V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$

- * $E = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_4, v_5\}, \{v_5, v_1\}, \{v_6, v_7\}, \{v_1, v_6\},$

- $\{v_2, v_6\}, \{v_4, v_6\}, \{v_5, v_6\}, \{v_2, v_7\}, \{v_3, v_7\}, \{v_4, v_7\}, \{v_5, v_7\}\}$

- Cota máxima de vértices permitida: $k = 5$.

- **Respuesta:** Si. Existe el subconjunto $W = \{v_1, v_3, v_4, v_6, v_7\}$ de tamaño $|W| = 5 \leq 5$. Se comprueba que todas las aristas del gráfica están cubiertas:

- **Aristas internas y centrales:** Todas las aristas que conectan hacia o entre los nodos centrales ($\{v_6, v_7\}, \{v_1, v_6\}, \{v_2, v_6\}, \dots$) quedan automáticamente cubiertas dado que $v_6, v_7 \in W$.

- **Aristas del ciclo exterior:**

- * $\{v_1, v_2\}$ se cubre por $v_1 \in W$.

- * $\{v_2, v_3\}$ se cubre por $v_3 \in W$.

- * $\{v_3, v_4\}$ se cubre por $v_3, v_4 \in W$.

- * $\{v_4, v_5\}$ se cubre por $v_4 \in W$.

- * $\{v_5, v_1\}$ se cubre por $v_1 \in W$.

Dado que toda arista en E posee al menos un extremo en W , el conjunto W constituye una cubierta de vértices válida de tamaño 5.

Definición en Forma de Optimización:

CUBIERTA DE VÉRTICES

- **Datos:** Una gráfica $G = (V, E)$.
- **Pregunta:** ¿Cuál es el subconjunto de vértices $W \subseteq V$ de cardinalidad mínima que cubre todas las aristas de G ? Es decir, encontrar [5]:

$$\min_{W \subseteq V} |W| \quad \text{sueto a} \quad \forall \{u, v\} \in E, \{u, v\} \cap W \neq \emptyset$$

3. Conjunto Independiente

Definición:

Finalmente, un *conjunto independiente* (o *conjunto estable*) es un subconjunto S de V tal que no hay dos vértices en S que sean adyacentes.

Definición en Forma Canónica:**CONJUNTO INDEPENDIENTE**

- **Entrada:** Una gráfica $G = (V, E)$.
- **Pregunta:** ¿Existe un subconjunto de vértices $S \subseteq V$ para G de mayor tamaño, tal que para todo par de vértices $u, v \in S$, no exista una arista $\{u, v\} \in E$?

Definición en Versión de Decisión:**CONJUNTO INDEPENDIENTE**

- **Entrada:** Una gráfica $G = (V, E)$ y un entero positivo $k \leq |V|$.
- **Pregunta:** ¿Existe un subconjunto de vértices $S \subseteq V$ de mayor tamaño $|S| \geq k$ tal que para todo par de vértices $u, v \in S$, no exista una arista $\{u, v\} \in E$?

Ejemplar concreto

- **Entrada:** Gráfica $G = (V, E)$ y un entero k donde:
 - $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$
 - $E = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_4, v_5\}, \{v_5, v_6\}\}$
 - Tamaño mínimo requerido: $k = 3$.
- **Respuesta:** Si. Existe el subconjunto $S = \{v_1, v_4, v_6\}$ de tamaño $|S| = 3 \geq 3$. Ninguna de las parejas formadas por sus elementos pertenece al conjunto de aristas E :

$$\{v_1, v_4\} \notin E, \quad \{v_1, v_6\} \notin E, \quad \{v_4, v_6\} \notin E$$

Por lo tanto, S cumple la condición de ser un conjunto independiente válido de tamaño 3.

Definición en Forma de Optimización:**CONJUNTO INDEPENDIENTE**

- **Entrada:** Una gráfica $G = (V, E)$.
- **Pregunta:** ¿Cuál es el subconjunto de vértices $S \subseteq V$ de tamaño máximo tal que ningún par de vértices en S sea adyacente entre sí?

Responde las siguientes preguntas sobre la gráfica no dirigida de la Figura ??:

- ¿Tiene un ciclo Hamiltoniano?

Respuesta: Si, existe un camino tal que, comenzando desde el vertice:

$$\{a1, a2, a4, a5...a19, a22, a1\}$$

, logra recorrer todos los vértices; por lo tanto, es verdad.

- ¿Cuál es el clan más grande?

Respuesta: Definimos a un clan, un subgrafo conexo, en el que el diámetro del subgrafo (el camino más corto de mayor longitud entre dos vértices cualesquiera dentro del grupo) no es mayor que un valor k . También hay que tener en cuenta la definición de clique, el cual una *clique* (o *clica*) es un subconjunto C de V tal que todos los pares de vértices en C son adyacentes.

Con esto en cuenta, el tamaño del clan máximo es 2. Para que exista un clan de tamaño 3 tendría que haber al menos un triángulo (K_3) en la gráfica (tres vértices donde todos se conecten entre sí). Por lo tanto, al no existir ningún triángulo (K_3). Así, cualquier par de vértices cumple la definición de clan; asimismo, tomamos el conjunto $\{a4, a5\}$

- ¿Cuál es el conjunto independiente más grande?

Respuesta: Se define como el conjunto independiente mas grande de una gráfica si y solo si no contiene un par de vértices adyacentes, el conjunto independiente mas grande seria: $I = \{a1, a2, a5, a7, a9, a11, a12, a15, a17, a19, a21\}$

- ¿Cuál es la cubierta de vértices más pequeña?

Respuesta: Se considera como un conjunto de elementos (vértices o aristas) que 'toca' o conecta a todos los elementos del otro tipo en la gráfica. Dado que toma el conjunto independiente de vértices que conecta a todos los elementos, entonces los vértices para la cubierta mas pequeña es en este caso equivalente al conjunto independiente del caso anterior. Pero la cubierta mínima no es el mismo conjunto independiente, sino su complemento, en ese caso:

$$V \setminus I = \{a_3, a_4, a_6, a_8, a_{10}, a_{13}, a_{14}, a_{16}, a_{18}, a_{20}, a_{22}\}$$

[5]

- Suponiendo que todas las aristas tienen peso 1, ¿Cuál es el corte máximo?

Respuesta:Suponiendo que el peso para cada arista es 1, la cardinalidad de $|V| = 22$ vértices ($a_1, a_2, \dots, a_{22}$) y aristas ($|E| = 33$). Una gráfica admite un corte que divida el total de sus aristas si y solo si es bipartito. [4]

Como una gráfica se considera bipartita cuando sus vértices (puntos o nodos) se pueden dividir en dos grupos separados de tal forma que las líneas o conexiones (aristas) solo van de un grupo al otro, de este modo existe una partición de los vértices en dos

conjuntos disjuntos V_1 y V_2 de tamaño $|V_1| = |V_2| = 11$, tales que no existen aristas con ambos extremos dentro de un mismo conjunto.

Dado que G es 3-regular (el numero de aristas para cada vértice es de grado 3). $3|V_1| = 3|V_2|$, y por tanto $|V_1| = |V_2|$. Como $|V_1| + |V_2| = 22$, se obtiene $|V_1| = |V_2| = 11$. [4]

Tomando el conjunto de vértices, denotamos a G como G_{22} dado el numero de vértices que posee. Para demostrar que G_{22} es bipartito, consideramos la siguiente partición de sus vértices:

$$V_1 = \{a_1, a_2, a_5, a_8, a_9, a_{11}, a_{12}, a_{14}, a_{15}, a_{20}, a_{21}\},$$

$$V_2 = \{a_3, a_4, a_6, a_7, a_{10}, a_{13}, a_{16}, a_{17}, a_{18}, a_{19}, a_{22}\}.$$

Se observa, a partir de las adyacencias para G_{22} , toda arista tiene un extremo en V_1 y el otro en V_2 . Por lo tanto,

$$E[V_1] = E[V_2] = \emptyset,$$

y, por definición, G_{22} es bipartito.

Asimismo que la gráfica G_{22} es bipartito, todas las 33 aristas conectan un vértice de V_1 con uno de V_2 . Por consiguiente, el valor del corte máximo coincide exactamente con la cardinalidad del conjunto de aristas:

$$\text{Max-Cut}(G_{22}) = w(V_1, V_2) = |E| = 33$$

Al tomar (V_1, V_2) como corte, todas las 33 aristas son cortadas. Como ningún corte puede contener más aristas que las existentes en el grafo, se concluye lo anterior. [6]

Ejercicio 5

Enunciado

Propón un algoritmo de búsqueda completa para alguno de los problemas (de decisión) del ejercicio anterior. Da el análisis de complejidad de tiempo e ilustra la ejecución con un ejemplar pequeño

Solución

El Concepto de Búsqueda Completa (*Backtracking*)

Un algoritmo de búsqueda completa para un problema de decisión en gráficas explora sistemáticamente el espacio de todos los subconjuntos posibles de vértices para encontrar uno que cumpla con la propiedad requerida.

Para **Vertex Cover**, el problema de decisión consiste en determinar: *Dada una gráfica $G = (V, E)$ y un entero k , ¿existe un subconjunto de vértices $S \subseteq V$ de tamaño $|S| \leq k$ que cubra todas las aristas de G ?*

Tenemos dos formas de plantear la búsqueda completa:

- **Fuerza Bruta:** Generar absolutamente todos los subconjuntos de tamaño k posibles en la gráfica y probar si alguno cubre todas las aristas. Hay $\binom{n}{k} \approx O(n^k)$ subconjuntos.
- **Backtracking:** Utilizar la estructura del problema para ramificar decisiones de manera inteligente. Este enfoque es sumamente superior y es el que explicaremos a continuación. Aunque de manera simplificada, pues muchas de las afirmaciones o puntos de partida sobre nuestro análisis están basados directamente de la sección **10.1 Finding Small Vertex Covers** de la literatura de Kleinberg y Tardos [7].

Propuesta del Algoritmo: Búsqueda Completa para *Vertex Cover*

Este algoritmo se basa en una observación clave: **Si seleccionamos cualquier arista $e = (u, v)$ de la gráfica, en cualquier cubierta de vértices válida, al menos uno de sus extremos (u o v) debe pertenecer a la cubierta.**

Esto nos permite diseñar un algoritmo recursivo de ramificación simple:

Pseudocódigo Formal:

Algoritmo BusquedaCompletaVC(G, k):

// $G = (V, E)$ es la gráfica actual, k es el presupuesto de vértices

1. Si G no tiene aristas:
Retornar VERDADERO (el conjunto vacío cubre todas las aristas)
2. Si G tiene más de $k * |V|$ aristas:
Retornar FALSO (un vértice de grado máximo $|V|-1$ solo puede cubrir a lo más $k*(|V|-1)$ aristas)
3. Si $k \leq 0$ y aún quedan aristas:
Retornar FALSO (nos quedamos sin presupuesto pero quedan aristas sin cubrir)
4. Seleccionar cualquier arista $e = (u, v)$ de la gráfica G
5. // Caso 1: Decidimos incluir al vértice ' u ' en la cubierta
 $G_u = G$ sin el vértice ' u ' y sin sus aristas incidentes
 $Rama_u = \text{BusquedaCompletaVC}(G_u, k - 1)$
6. // Caso 2: Decidimos incluir al vértice ' v ' en la cubierta

```

G_v = G sin el vértice 'v' y sin sus aristas incidentes
Rama_v = BusquedaCompletaVC(G_v, k - 1)

// Si alguna de las ramas tiene éxito, hay solución
7. Retornar (Rama_u 0 Rama_v)

```

Análisis de Complejidad Temporal

Para analizar la complejidad de este algoritmo, debemos comprender su **árbol de recursión**:

1. **Profundidad del Árbol:** En cada llamada recursiva que ramifica, el valor de k disminuye en exactamente 1 unidad ($k - 1$). Por lo tanto, el árbol de llamadas recursivas tiene una profundidad máxima de k .
2. **Número de Nodos (Llamadas):** Como cada nodo que no es una hoja se divide en exactamente 2 llamadas (una para u y otra para v), el número máximo de llamadas en el árbol es la suma de una progresión geométrica:

$$1 + 2 + 4 + 8 + \dots + 2^k = 2^{k+1} - 1 = O(2^k)$$

3. **Costo en cada Nodo:** En cada llamada recursiva, necesitamos verificar si quedan aristas, contar aristas, seleccionar una arista y crear una nueva subgráfica eliminando un vértice. Con una representación adecuada (como listas de adyacencia), esto se puede realizar en tiempo lineal respecto a los vértices, es decir, $O(|V|)$ o $O(|V| + |E|)$.

Complejidad Total:

La complejidad temporal total es:

$$O(2^k \cdot |V|)$$

Mientras que la fuerza bruta toma tiempo polinomial de alto grado con exponente variable $O(|V|^k)$, este algoritmo mantiene la base exponencial dependiente exclusivamente de k (2^k) y el tamaño de la gráfica entra de manera estrictamente lineal ($|V|$). Esto hace que sea perfectamente viable para valores de k pequeños (como $k \leq 15$), incluso si la gráfica tiene miles de vértices.

Ilustración del Algoritmo

Ejercicio 6

Enunciado

Supón que un algoritmo A, que resuelve un problema Π , tiene un tiempo de ejecución de $T_A(n) = 20n^3 + 10n + 16$, además supón que un algoritmo B, que resuelve el mismo problema Π , tiene un tiempo de ejecución de $T_B(n) = 22n^2 \log_2(8n)$.

1. ¿Qué algoritmo es mejor?
2. ¿Para qué valores de n el algoritmo A es mejor que el B?
3. ¿Para qué valores de n el algoritmo B es mejor que el A?

Solución

1. ¿Qué algoritmo es mejor? Tenemos lo siguiente:

$$\begin{aligned}
 \lim_{n \rightarrow \infty} \frac{T_A(n)}{T_B(n)} &= \lim_{n \rightarrow \infty} \frac{20n^3 + 10n + 16}{22n^2(3 + \log_2 n)} \\
 &= \lim_{n \rightarrow \infty} \frac{20n^3 \left(1 + \frac{10}{20n^2} + \frac{16}{20n^3}\right)}{22n^2 \log_2 n \left(1 + \frac{3}{\log_2 n}\right)} \\
 &= \lim_{n \rightarrow \infty} \left(\frac{20}{22}\right) \cdot \frac{n^3}{n^2 \log_2 n} \cdot \frac{1 + \frac{10}{20n^2} + \frac{16}{20n^3}}{1 + \frac{3}{\log_2 n}} \\
 &= \frac{10}{11} \lim_{n \rightarrow \infty} \frac{n^3}{n^2 \log_2 n} \cdot \frac{1 + 0 + 0}{1 + 0} \\
 &= \frac{10}{11} \lim_{n \rightarrow \infty} \frac{n}{\log_2 n} \stackrel{\text{L'H}}{=} \frac{10}{11} \lim_{n \rightarrow \infty} n \ln 2 = \infty
 \end{aligned}$$

Por la definición de ω -notación, el algoritmo B es mejor.

Tabulamos valores de n para ambos algoritmos:

n	$T_A(n) = 20n^3 + 10n + 16$	$T_B(n) = 22n^2 \log_2(8n)$	Mejor algoritmo
1	46	66	A
2	196	352	A
3	586	≈ 907.82	A
4	1336	1760	A
5	2566	≈ 2927.06	A
6	4396	≈ 4423.29	A
7	6946	≈ 6260.33	B
8	10336	8448	B
9	14686	≈ 10994.80	B

Para confirmar que para todo $n \geq 7$ el algoritmo B es mejor que el A , consideramos la función

$$d(n) = T_A(n) - T_B(n)$$

y mostramos que $d(n)$ es estrictamente creciente para todo $n \geq 7$.

Derivamos d con respecto a n :

$$d'(n) = 60n^2 + 10 - 22n \left(2 \log_2(8n) + \frac{1}{\ln 2} \right)$$

Dado que $T'_A(n)$ crece cuadráticamente y domina sobre el crecimiento lineal-logarítmico de $T'_B(n)$, se cumple que $d'(n) > 0$. Por el criterio de la primera derivada, $d(n)$ es estrictamente creciente y como $d(7) = 6946 - 6260.33 \approx 685.67 > 0$, se sigue por monotonía que:

$$d(n) \geq d(7) > 0, \quad \forall n \geq 7$$

Por lo tanto, $T_A(n) > T_B(n)$ para todo $n \geq 7$.

2. Valores de n tal que A es mejor:

$$\{n \in \mathbb{N} \mid 1 \leq n \leq 6\}$$

3. Valores de n tal que B es mejor:

$$\{n \in \mathbb{N} \mid n \geq 7\}$$

Ejercicio 7

Enunciado

Para cada par de expresiones $f(n)$ y $g(n)$ indica si f es O, Ω, Θ de $g(n)$. Asume que $k \geq 1$ y $c > 1$. Hay que llenar la tabla con un sí o un no en cada celda, pero se debe agregar la justificación para cada caso.

f	g	O	Ω	Θ
$100n^2 + 3n$	$n^3 + 1$			
n^k	c^n			
2^n	$2^{n/2}$			
$\log n$	$\sqrt{n}$			
$n!$	2^n			

* Resolver al menos un inciso con límites, y al menos uno utilizando las definiciones explícitas de la Notación Asintótica correspondiente.

Solución

$f(n)$	$g(n)$	O	Ω	Θ
$100n^2 + 3n$	$n^3 + 1$	Sí	No	No
n^k	c^n	Sí	No	No
2^n	$2^{n/2}$	No	Sí	No
$\log n$	$\sqrt{n}$	Sí	No	No
$n!$	2^n	No	Sí	No

Usamos las definiciones de O, o, ω y Ω para cada ejercicio.

1. $f(n) = 100n^2 + 3n$ y $g(n) = n^3 + 1$

- ¿ $f(n) = O(g(n))$?: **Sí**. Buscamos constantes $c > 0$ y $n_0 \in \mathbb{N}$ tales que:

$$0 \leq 100n^2 + 3n \leq c(n^3 + 1), \quad \forall n \geq n_0$$

Para todo $n \geq 1$, se cumple:

$$100n^2 + 3n \leq 100n^2 + 3n^2 = 103n^2 \leq 103n^3 \leq 103(n^3 + 1)$$

Tomando $c = 103$ y $n_0 = 1$:

$$\exists c > 0, \exists n_0 \in \mathbb{N} : 0 \leq 100n^2 + 3n \leq c(n^3 + 1), \quad \forall n \geq n_0 \implies f(n) = O(g(n))$$

- $\mathfrak{L}f(n) = \Omega(g(n))$?: **No.** Supongamos por contradicción que $f(n) = \Omega(g(n))$, es decir:

$$\exists c > 0, \exists n_0 \in \mathbb{N} : 0 \leq c(n^3 + 1) \leq 100n^2 + 3n, \quad \forall n \geq n_0$$

Se deriva lo siguiente para todo $n \geq 1$ y $n \geq n_0$:

$$\begin{aligned} c(n^3 + 1) \leq 100n^2 + 3n &\implies c(n^3 + 1) \leq 100n^2 + 3n^2 = 103n^2 \\ &\implies n^3 \leq n^3 + 1 \leq \frac{103n^2}{c} \\ &\implies n \leq \frac{103}{c} \quad \text{— una contradicción.} \end{aligned}$$

Por tanto, $f(n) \neq \Omega(g(n))$.

- $\mathfrak{L}f(n) = \Theta(g(n))$?: **No.** De ambos resultados se sigue que $f(n) \neq \Theta(g(n))$.

2. $f(n) = n^k$ y $g(n) = c^n$

- $\mathfrak{L}f(n) = O(g(n))$?: **Sí.** Evaluamos el límite aplicando la regla de L'Hôpital k veces:

$$\lim_{n \rightarrow \infty} \frac{n^k}{c^n} = \lim_{n \rightarrow \infty} \frac{\frac{d^k}{dn^k} n^k}{\frac{d^k}{dn^k} c^n} = \lim_{n \rightarrow \infty} \frac{k!}{(\ln c)^k c^n} = 0$$

Dado que el límite es 0, se tiene que $f(n) = o(g(n)) \implies f(n) = O(g(n))$.

- $\mathfrak{L}f(n) = \Omega(g(n))$?: **No.** Por definición de o -notación y Ω -notación:

$$f(n) = o(g(n)) \implies f(n) \neq \Omega(g(n))$$

- $\mathfrak{L}f(n) = \Theta(g(n))$?: **No.** De ambos resultados se sigue que $f(n) \neq \Theta(g(n))$.

3. $f(n) = 2^n$ y $g(n) = 2^{n/2}$

- $\mathfrak{L}f(n) = \Omega(g(n))$?: **Sí.** Evaluamos el límite siguiente:

$$\lim_{n \rightarrow \infty} \frac{2^n}{2^{n/2}} = \lim_{n \rightarrow \infty} 2^{n-n/2} = \lim_{n \rightarrow \infty} 2^{n/2} = \infty$$

Se concluye que $f(n) = \omega(g(n)) \implies f(n) = \Omega(g(n))$.

- $\mathfrak{L}f(n) = O(g(n))$?: **No.** Por definición de ω -notación y O -notación:

$$f(n) = \omega(g(n)) \implies f(n) \neq O(g(n))$$

- $\mathfrak{L}f(n) = \Theta(g(n))$?: **No.** De ambos resultados se sigue que $f(n) \neq \Theta(g(n))$.

4. $f(n) = \log n$ y $g(n) = \sqrt{n} = n^{1/2}$

- $\mathfrak{L}f(n) = O(g(n))$?: **Sí.** Evaluamos el límite aplicando la regla de L'Hôpital:

$$\lim_{n \rightarrow \infty} \frac{\log n}{n^{1/2}} = \lim_{n \rightarrow \infty} \frac{\frac{d}{dn} \log n}{\frac{d}{dn} n^{1/2}} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n \ln 2}}{\frac{1}{2} n^{-1/2}} = \frac{2}{\ln 2} \lim_{n \rightarrow \infty} \frac{1}{n \cdot n^{-1/2}} = \frac{2}{\ln 2} \lim_{n \rightarrow \infty} \frac{1}{n^{1/2}} = 0$$

Dado que el límite es 0, se cumple que $f(n) = o(g(n)) \implies f(n) = O(g(n))$.

- ¿ $f(n) = \Omega(g(n))$?: **No.** Por definición de o -notación y Ω -notación:

$$f(n) = o(g(n)) \implies f(n) \neq \Omega(g(n))$$

- ¿ $f(n) = \Theta(g(n))$?: **No.** De ambos resultados se sigue que $f(n) \neq \Theta(g(n))$.

5. $f(n) = n!$ y $g(n) = 2^n$

- ¿ $f(n) = \Omega(g(n))$?: **Sí.** Expandimos el cociente y observamos que $\frac{n}{2} \geq 2$ para todo $n \geq 4$:

$$\frac{n!}{2^n} = \left(\frac{1}{2} \cdot \frac{2}{2} \cdot \frac{3}{2}\right) \left(\frac{4}{2} \cdot \frac{5}{2} \cdots \frac{n}{2}\right) \geq \frac{3}{4} \cdot 2^{n-3}$$

Evaluamos el límite

$$\lim_{n \rightarrow \infty} \left(\frac{3}{4} \cdot 2^{n-3}\right) = \frac{3}{4} \lim_{n \rightarrow \infty} 2^{n-3} = \infty$$

Usando límite evaluado y la desigualdad planteada tenemos que

$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \infty \implies f(n) = \omega(g(n)) \implies f(n) = \Omega(g(n))$$

- ¿ $f(n) = O(g(n))$?: **No.** Por definición de ω -notación y O -notación:

$$f(n) = \omega(g(n)) \implies f(n) \neq O(g(n))$$

- ¿ $f(n) = \Theta(g(n))$?: **No.** De ambos resultados se sigue que $f(n) \neq \Theta(g(n))$. [8]

Referencias Bibliográficas

Referencias

- [1] Herrera Herrera, A. A., Méndez López, D. A., & Cisneros Díaz, I. A. (2025). La función Lambert W: teoría, propiedades y aplicaciones. *Ciencias Matemáticas*, 39(1), 11–19. DOI: <https://doi.org/10.5281/zenodo.17443639>. Recuperado a partir de <https://revistas.uh.cu/rcm/article/view/10777>.
- [2] Pensando Numéricamente. (2021). *FUNCIÓN W de LAMBERT: Origen, propiedades, dominio y aplicaciones / Pensando Numéricamente* [Video]. YouTube. https://www.youtube.com/watch?v=HU5_qbUqKBU
- [3] S. Poljak y F. Rendl. Solving the max-cut problem using eigenvalues. *Discrete Applied Mathematics*, 62(1):249–278, 1995.
- [4] J. A. Bondy y U. S. R. Murty. *Graph Theory*. Graduate Texts in Mathematics, vol. 244. Springer, London, 2008.

-
- [5] D. Jungnickel. *Graphs, Networks and Algorithms*. Editado por M. Bronstein, A. M. Cohen, H. Cohen, D. Eisenbud y B. Sturmfels. Algorithms and Computation in Mathematics, vol. 5. Springer, Berlin, Heidelberg, 2005.
 - [6] W. Goddard y M. A. Henning. Independent domination in graphs: A survey and recent results. *Discrete Mathematics*, 313(7):839–854, 2013.
 - [7] Kleinberg, J., & Tardos, É. (2005). *Algorithm Design* (1st ed.). Pearson/Addison-Wesley. (Específicamente Capítulo 7: Network Flow).
 - [8] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). Cambridge, MA: MIT Press.